

Architettura dei Calcolatori

Primo appello – 9 giugno 2021 – 1h45

PARTE 1 – RISPOSTA SINGOLA - Ogni domanda ha una sola risposta VERA.

- Una risposta esatta fa acquisire il punteggio positivo riportato a fianco della domanda
- Una risposta errata fa perdere il punteggio negativo riportato a fianco della domanda
- Una risposta lasciata in bianco viene valutata 0

1. **[2, -0.5]** Si consideri di eseguire il seguente stralcio di codice su una pipeline RISC-V con logica di forwarding e con dynamic branch predictor a 1 bit. Inizialmente il predittore è settato a BRANCH TAKEN. Qual è la misprediction rate per questo codice?

```
li    t0, 1
li    t1, 0
li    t2, 10
loop:
    slli t0, t0, 1
    addi t1, t1, 1
    blt  t1, t2, loop
```

- a) 10%
b) 20%
c) 30%
d) Nessuna delle precedenti
2. **[3, -0.5]** Si consideri un certo programma con 70% di istruzioni di tipo aritmetico, 10% load/store e 20% branch. Si assuma che le istruzioni aritmetiche eseguano in due cicli. Il sistema ha un solo livello di cache, con hit rate pari al 90% (hit=1ciclo, miss=50 cicli). Il sistema usa dynamic branch prediction, con misprediction rate del 10% (predizione corretta=1 ciclo; predizione scorretta = 5 cicli). Qual è il CPI medio?
- a) 1,2
b) 2,6
c) 3,7
d) Nessuna delle precedenti
3. **[3, -0.5]** L'istruzione blt x2, x3, 0x7F0
- a) Si codifica in binario come 0111 1110 0011 0001 0100 0000 1110 0111
b) Si codifica in binario come 0111 1110 0010 0001 1100 1000 0110 0111
c) Si codifica in esadecimale come 0x7E21C867
d) Nessuna delle precedenti

PARTE 2 – (POSSIBILI) RISPOSTE MULTIPLE -
Ogni domanda può avere da una a quattro risposte CORRETTE.

- Ogni risposta esatta viene calcolata: +1
 - Ogni risposta errata viene calcolata: -0.5
 - Una risposta lasciata in bianco viene calcolata: 0
4. Una CPU RISC-V 32bit usa una L1 cache direct mapped grande 256B e con linee da 8 word. Quali affermazioni sono vere?
- a) Ogni linea può ospitare 2^{19} diversi indirizzi
 - b) A cache fredda due accessi successivi agli indirizzi 0x0020 e 0x0033 generano rispettivamente una compulsory miss e una hit
 - c) A seguito degli accessi al punto b) due nuovi accessi agli indirizzi 0x0000 e 0x2021 generano rispettivamente una compulsory miss e una conflict miss
 - d) A seguito degli accessi al punto b) e c) un nuovo accesso all'indirizzo 0x14 genera una compulsory miss
5. Se interpretata come un'istruzione RISC-V la stringa 0x00450513
- a) Equivale all'istruzione addi x10, x10, 4
 - b) Utilizza un immediato
 - c) Ha i bit del campo immediato disordinati
 - d) Ha 20 bit di immediato
6. Si consideri il seguente stralcio di codice RISC-V, scritto per eseguire su una pipeline priva di logica di detection dei data hazards e forwarding:

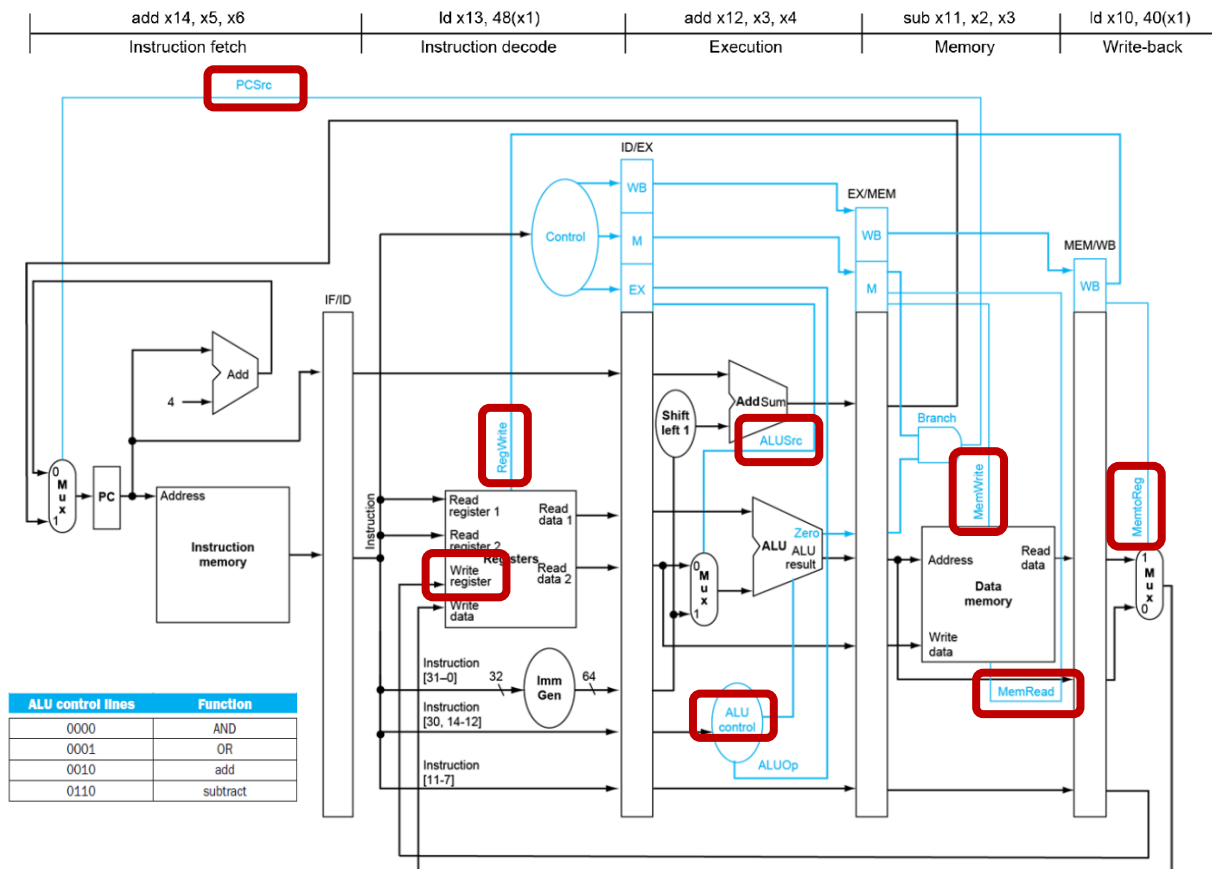
```
slli    t0, t0, 2
nop
nop
add     t0, t0, x10
add     t1, x11, x12
sub     t2, x13, x14
nop
nop
mul     t1, t1, t2
nop
nop
add     t1, t0, t1
ld      t2, 0(t0)
nop
nop
sd      t2, 0(t1)
```

- a) Il codice esegue in 20 cicli (considerando la pipeline inizialmente vuota)
- b) È possibile riordinare il codice per ridurre il numero di NOP. Così facendo si riduce il numero di cicli a 16
- c) Se eseguito su una pipeline dotata di logica di detection e forwarding dei data hazards il codice non richiede NOP
- d) Non è possibile riordinare il codice per ridurre il numero di NOP

PARTE 3 – DOMANDE APERTE

- Una risposta esatta fa acquisire il punteggio positivo riportato a fianco della domanda
- Una risposta errata può causare una penalità che dipende dalla gravità dell'errore
- Una risposta lasciata in bianco viene calcolata: 0

7. [6 pt] Si consideri il seguente diagramma single-cycle



relativo all'esecuzione del seguente stralcio di codice assembly:

```
ld    x10, 40(x1)
sub   x11, x2, x3
add   x12, x3, x4
ld    x13, 48(x1)
add   x14, x5, x6
```

Dire quanto valgono i segnali cerchiati in figura, motivando chiaramente le risposte.

8. [6 pt] Progettare una rete sequenziale che comanda il circuito sonoro di un giocattolo per bambini capace di emettere due diverse sequenze di tre note (DO, MI, SOL) selezionabili tramite un interruttore I tale che:

– se $I = 0$ le tre note suonano individualmente, una dopo l'altra, cambiando ad ogni ciclo del segnale di clock che governa il circuito:

(ciclo 0) DO

(ciclo 1) MI

(ciclo 2) SOL

(ciclo 3) DO

(ciclo 4) MI

...

– se $I = 1$ la melodia è armonizzata a bicordi (ovvero, suonano due note contemporaneamente per volta), sempre in sequenza secondo il segnale di clock:

(ciclo 0) DO MI

(ciclo 1) MI SOL

(ciclo 2) SOL DO

(ciclo 3) DO MI

(ciclo 4) MI SOL

...

Ad ogni pressione dell'interruttore la sequenza selezionata ricomincia a suonare da capo.

Dire se conviene utilizzare un automa di Mealy o di Moore, motivando chiaramente la risposta. Ricavare la specifica degli stati, il diagramma di transizione, le tabelle di verità e le forme minime per le reti di stato futuro e delle uscite.

9. [6 pt] Si scriva una funzione **sommatoria** che prende in ingresso un intero n e ritorna la sommatoria mostrata sotto. Se n è non positivo la funzione ritorna -1, se è nullo ritorna 0.

$$1^1 + 2^2 + 3^3 + \dots + n^n$$

Per svolgere il calcolo la funzione **sommatoria** deve chiamare una funzione **elev** che prende in ingresso un intero a e restituisce a^a .

Si scrivano le funzioni **sommatoria** e **elev** rispettando la calling convention RISC-V e gestendo opportunamente lo stack e il register file, utilizzando i registri appropriati.

Appendice: Tabelle di codifica delle istruzioni RISC-V e registri

Format	Instruction	Opcode	Funct3	Funct6/7
R-type	add	0110011	000	0000000
	sub	0110011	000	0100000
	sll	0110011	001	0000000
	xor	0110011	100	0000000
	srl	0110011	101	0000000
	sra	0110011	101	0000000
	or	0110011	110	0000000
	and	0110011	111	0000000
	lr.d	0110011	011	0001000
	sc.d	0110011	011	0001100
Format	Instruction	Opcode	Funct3	Funct6/7
I-type	lb	0000011	000	n.a.
	lh	0000011	001	n.a.
	lw	0000011	010	n.a.
	ld	0000011	011	n.a.
	lbu	0000011	100	n.a.
	lhu	0000011	101	n.a.
	lwu	0000011	110	n.a.
	addi	0010011	000	n.a.
	slli	0010011	001	0000000
	xori	0010011	100	n.a.
	srlr	0010011	101	0000000
	srai	0010011	101	0100000
	ori	0010011	110	n.a.
	andi	0010011	111	n.a.
	jalr	1100111	000	n.a.
Format	Instruction	Opcode	Funct3	Funct6/7
S-type	sb	0100011	000	n.a.
	sh	0100011	001	n.a.
	sw	0100011	010	n.a.
	sd	0100011	111	n.a.
SB-type	beq	1100111	000	n.a.
	bne	1100111	001	n.a.
	blt	1100111	100	n.a.
	bge	1100111	101	n.a.
	bltu	1100111	110	n.a.
	bgeu	1100111	111	n.a.
U-type	lui	0110111	n.a.	n.a.
UJ-type	jal	1101111	n.a.	n.a.

Name (Field Size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format